



Better shifting

Document number:

[P3793R1](#)

Date:

2025-11-19

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Brian Bi <bbi5291@gmail.com>

Co-authors:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3793/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/better-shifting.cow



ABSTRACT: We propose the addition of functions to the `<bit>` header to perform bit shifts on integer operands. The `std::shl` and `std::shr` functions provide the following advantages over the built-in shift operators:

1. They always produce a mathematically correct result when possible.
2. They never have undefined behavior.
3. They avoid the confusing precedence of the built-in shift operators.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R0
- 2 **Introduction**
- 3 **Motivation — Why are overlong shifts needed?**
- 4 **Design considerations**
 - 4.1 Proposed behavior for overlong shifts



4.1.1	Wrapping behavior is not useful
4.1.2	Shift amounts which are overlong by more than one should be safe
4.2	Logical versus arithmetic shifts
4.3	Negative shift amounts
4.3.1	Emulating possible behaviors using <code>std::shl</code>
4.3.2	Benchmarks of <code>std::shl</code> variants
4.4	Naming
4.5	Signatures
4.6	SIMD support
5	Possible implementation
6	Wording
6.1	[version.syn]
6.2	[bit]
6.3	[simd]
7	References

§ 1. Revision history

§ 1.1. Changes since R0

- changed the function signatures to take any signed or unsigned `S` instead of `int`
- changed behavior for negative shift amounts to "shift in the opposite direction"
- added §4.3.1. [Emulating possible behaviors using `std::shl`](#)
- added §4.3.2. [Benchmarks of `std::shl` variants](#)
- added §5. [Possible implementation](#)
- rebased §6. [Wording](#) on [N5014]
- fixed various minor typos and wording issues

§ 2. Introduction

C++ has built-in shift operators, `<<` and `>>`, inherited from C with semantics essentially unchanged, including the following two inconvenient properties:

1. The precedence of these operators is less than the precedence of the additive operators, which is counterintuitive because shift operations behave as multiplication and division by a power of 2.
2. If the shift amount is greater than or equal to the width of the (promoted) left operand or is negative, the behavior is undefined. In the remainder of this document, we will refer to

shifts by a positive amount that is greater than or equal to the bit width of the operand as *overlong* shifts.



The first property certainly cannot be changed at this point in time. Reflector discussion revealed that GCC relies on the second property when vectorizing multiple shift operations on adjacent memory locations, and changing it might therefore not be "free". Besides that, changing the second property is also likely to be contentious because some committee members prefer to make overlong shifts erroneous behavior instead of defining them to produce the mathematically correct result.

For these reasons, we are instead proposing to solve the problems with shifting in C++ by introducing new Standard Library facilities in the `<bit>` header.

§ 3. Motivation — Why are overlong shifts needed?

We consider the undefined behavior of overlong shifts to be gratuitous. Unlike other arithmetic operators, which produce undefined behavior only when the result is "not mathematically defined or is outside the range of representable values" ([\[expr.pre\] paragraph 4](#)), the built-in shift operators unconditionally produce UB for overlong shifts. This behavior is inherited from C, in which the arithmetic operators were originally designed to do whatever the corresponding hardware instructions would do; processor families differ as to how overlong shifts are handled. However, it is unclear why the behavior of overlong shifts was standardized as being undefined behavior as opposed to producing an unspecified result; perhaps there's some CPU that we (the authors of this paper) don't know about (and that might even be obsolete) on which the behavior could include trapping, halting, or otherwise failing to produce a result.

Besides that, there are practical reasons why the undefined behavior of overlong shifts is inconvenient, particularly when the second operand is equal to the bit width of the first operand.



EXAMPLE: Consider the task of implementing the following function:

```
/// Return the value of x with the least significant num_bits bits masked out.  
/// num_bits shall be nonnegative and ≤ 32.  
std::uint32_t mask_lsb(std::uint32_t x, int num_bits);
```

When `num_bits` is equal to 32, the behavior of `mask_lsb` is the natural continuation of the behavior for smaller values of `num_bits`. However, to implement this function, we must guard against `num_bits == 32`:

```
std::uint32_t mask_lsb(std::uint32_t x, int num_bits) {  
    if (num_bits == 32) return x;  
    return x & ~((static_cast<std::uint32_t>(1) << num_bits) - 1);  
}
```

The first statement in the body of `mask_lsb` would be unnecessary if the shift operation produced the mathematically correct result of `0` when `num_bits` is equal to `32`.



Overlong bit-shifts can also be a problem with right-shifting, although these problems are significantly less common.



EXAMPLE: We can use an integer `b` to represent a bitset, where `1` represents an element which is in the set, and `0` represents an integer that does not.

```
std::uint32_t bitset = /* ... */;

// Possibly undefined behavior; would require special cases to guard against overlong shifts.
bool contains(size_t i) {
    return (bitset >> i) & 1;
}

// Never undefined.
bool contains(size_t i) noexcept {
    return std::shr(bitset, int(i)) & 1;
}
```

Reporting an element that `bitset` has no capacity for as not being in the set may be exactly the behavior we want, rather than UB, and `std::shr` makes that easy.

§ 4. Design considerations

§ 4.1. Proposed behavior for overlong shifts

We propose the addition of Standard Library functions `std::shl` and `std::shr` that produce mathematically correct results for overlong shifts. That is, these functions actually just shift the bits by the number of positions specified: in the case of an overlong shift, that means that all bits that would be shifted are shifted off the end. For a logical shift, the result is `0`; for an arithmetic right shift, the result is `-1` when the first operand is negative, and `0` otherwise.

§ 4.1.1. Wrapping behavior is not useful

We believe that the "wrapping" behavior (most famously exhibited by the x86 family of processors) in which the shift amount is reduced modulo the bit width of the other operand is not useful, other than when implementing bit rotations. Since the functions `std::rotr` and `std::rotr` are already provided by the Standard Library, C++ programmers do not need to implement rotations themselves anymore, and would not benefit from wrapping behavior for shifts.

§ 4.1.2. Shift amounts which are overlong by more than one should be safe

Although shifts by an amount that are strictly greater than the bit width of the first operand are not as useful as when the amount is equal to the bit width, we believe that requiring the

implementation to produce the mathematically correct result in those cases does not impose an additional performance burden. For example, if `x` is 32 bits wide, then requiring `std::shl(x, 33)` to produce 0 does not impose additional overhead beyond only requiring `std::shl(x, 32)` to produce 0.



EXAMPLE: For example, GCC 15.1 at -O2 or higher produces very similar x86-64 assembly for the following two functions:

```
unsigned int shl1(unsigned int x, unsigned int amount) {
    if (amount < 32) return x << amount; else return 0;
}
unsigned int shl2(unsigned int x, unsigned int amount) {
    if (amount > 32) std::unreachable();
    if (amount < 32) return x << amount; else return 0;
}
```

```
shl1:
    mov     ecx, esi
    xor     eax, eax
    sal     edi, cl
    cmp     esi, 32
    cmovb   eax, edi
    ret

shl2:
    mov     ecx, esi
    xor     eax, eax
    sal     edi, cl
    cmp     esi, 32
    cmovne  eax, edi
    ret
```

Using `shl2` instead of `shl1` can improve performance only in the sense that UB enables more optimizations in general (e.g. assuming that the branch leading to the UB is not taken). The generated code in Clang 20 is very similar.

Such similarities in generated assembly are observed across many other architectures because if a single branch or conditional move instruction is required in order to produce the mathematically correct result for `amount >= 32`, then a single one will also be needed merely to account for the case of `amount == 32` (when `amount > 32` is disallowed).

§ 4.2. Logical versus arithmetic shifts

The built-in `>>` operator performs an arithmetic shift on signed operands: that is, the sign bit is extended. One possible design is to provide both arithmetic and logical right shifts, either as separate functions or as one function with an additional parameter indicating the choice of arithmetic or logical shift.

However, we believe that this is unnecessary because in cases where the programmer wishes to always perform a logical shift, it is customary to employ unsigned types, possibly by inserting a

cast that will be optimized out. Conversely, the deliberate choice to use a signed type for the left operand of a right shift indicates intent to perform an arithmetic shift. The proposed `std::to_signed` and `std::to_unsigned` functions in [P3643R0] would make such conversions convenient, even in generic code.

A survey of popular programming languages supports this design direction:

- Languages that provide an explicit choice between a logical shift or an arithmetic shift usually lack unsigned integer types (Fortran, Java, JavaScript, OCaml). C# had unsigned integer types from its initial release, but didn't have the logical right shift operator until version 11.
- Languages that do not provide an explicit choice between a logical shift and an arithmetic shift always perform arithmetic shifts for negative operands (Go, Haskell, Objective-C, Python, Ruby, Rust).
- Perl is possibly an exception to the above, but it is hard to track down the behavior of shifts prior to version 5.

§ 4.3. Negative shift amounts

The behavior of negative shift amounts in some popular programming languages is listed below:

- Shift amount reduced modulo bit width of other operand: C#, Java, JavaScript
- Exception or panic: Go, Haskell, Python
- Shift in other direction: Fortran, Objective-C, Perl, Ruby
- Unspecified or implementation-defined result: OCaml, Rust

The mathematically correct behavior is to "shift in the other direction". That is if we consider left-shifting to be a multiplication with a power of two; a negative shift is a negative exponent, i.e. a division by a power of two, i.e. a right-shift. **We propose this "mathematically correct" behavior.**

R0 of this paper proposed an implementation-defined result and erroneous behavior. Possible alternatives were discussed in SG6, and neither the proposed direction nor the alternatives sparked confidence. However, we now believe we've identified the right direction for the following reasons:

- The implementation-defined result in the R0 `std::shl` was perceived as a "half-measure".
- Shifting in the opposite direction provides a clear distinction in behavior from `<<`. It is as safe as costly as possible, whereas `<<` is as unsafe and cheap as possible.
- All other behaviors can be obtained by modifying the call site (§4.3.1. [Emulating possible behaviors using `std::shl`](#)).
- The performance cost of shifting in the opposite direction was measured to be acceptable (§4.3.2. [Benchmarks of `std::shl` variants](#)).

§ 4.3.1. Emulating possible behaviors using `std::shl`



EXAMPLE: See below various different ways to use `std::shl`, from most safe (top) to least safe (bottom):



```
int32_t x = /* ... */;
int32_t s = /* ... */;

// If s is negative, shift in the opposite direction.
int32_t r = std::shl(x, s);

// If s is negative, r is meaningless but well-defined.
// This is faster because no check for negatives is necessary.
int32_t r = std::shl(x, uint32_t(s));

// If s is negative, r is meaningless, and a contract violation occurs.
// This is R0 of this paper, except we used EB, not contracts.
contract_assert(s >= 0);
int32_t r = std::shl(x, uint32_t(s));

// If s is negative, the behavior is undefined.
[[assume(s >= 0)]];
int32_t r = std::shl(x, s);

// Equivalent to x << s.
[[assume(s >= 0 && s < 32)]];
int32_t r = std::shl(x, s);
```

§ 4.3.2. Benchmarks of `std::shl` variants

To make an educated choice about which behavior would be appropriate for negative shifts in `std::shl`, find a comparison of the following functions below ([QuickBench], [CompilerExplorer]):



NOTE: The latter functions described as "bidirectional" shift in the opposite direction for negative shift amounts. The `shl_branchless` and `shl_branching` functions display behavior that was originally proposed (meaningless result for negative shifts).

Some of the implementations rely on behavior that is *technically* undefined, but "works in practice".

Function	Assembly (clang++-21 -O2 -march=x86-64-v4)	Time
<pre>unsigned shl_unsafe(unsigned x, int s) { return x << s; }</pre>	<pre>shl_unsafe: shlx eax, edi, esi ret</pre>	1.0 ×
<pre>unsigned shl_branchless(unsigned x, int s) {</pre>	<pre>shl_branchless: xor ecx, ecx cmp esi, 32 shlx eax, edi, esi</pre>	1.2 ×

<pre> return (s < 32) * (x << s); } </pre>	<pre> cmovge eax, ecx ret </pre>	
<pre> unsigned shl_branching(unsigned x, int s) { if (s < 32) return x << s; else return 0; } </pre>	<pre> shl_branching: xor ecx, ecx cmp esi, 32 shlx eax, edi, esi cmovge eax, ecx ret </pre>	1.2 ×
<pre> // Bidirectional (proposed behavior). unsigned shl_bi_branching(unsigned x, int s) { if (s >= 0) { if (s < 32) return x << s; else return 0; } else { if (s > -32) return x >> -s; else return 0; } } </pre>	<pre> shl_bi_branching: test esi, esi js .LBB4_2 xor ecx, ecx cmp esi, 32 shlx eax, edi, esi cmovae eax, ecx ret .LBB4_2: xor eax, eax cmp esi, -31 jb .LBB4_4 neg sil shrx eax, edi, esi .LBB4_4: ret </pre>	1.6 ×
<pre> // Bidirectional (proposed behavior). unsigned shl_bi_semi_branchless(unsigned x, int s) { if (s >= 0) { return (s < 32) * (x << s); } else { return (s > -32) * (x >> -s); } } </pre>	<pre> shl_bi_semi_branchless: test esi, esi js .LBB5_2 xor ecx, ecx cmp esi, 32 shlx eax, edi, esi cmovae eax, ecx ret .LBB5_2: mov eax, esi neg al xor ecx, ecx cmp esi, -31 shrx eax, edi, eax cmovb eax, ecx ret </pre>	1.6 ×
<pre> // Bidirectional (proposed behavior). unsigned shl_bi_branchless(unsigned x, int s) { unsigned pos = (x < 32) * (x << s); unsigned neg = (x > -32) * (x >> -s); int mask = s >> 31; return (mask & neg) (~mask & pos); } </pre>	<pre> shl_bi_branchless: xor ecx, ecx cmp edi, 32 shlx eax, edi, esi cmovae eax, ecx mov edx, esi neg dl shrx edx, edi, edx mov r8d, esi sar r8d, 31 and r8d, edx cmp edi, -31 </pre>	2.9 ×

	<code>cmovb r8d, ecx</code> <code>test esi, esi</code> <code>cmovs eax, ecx</code> <code>or eax, r8d</code> <code>ret</code>	
--	--	---

The key takeaways are:

- Handling of negative shift amounts incurs roughly a 25% performance cost relative to producing a meaningless result, and a 60% cost relative to the builtin `<<` operator.
- Attempts at manual branchless optimization provide little to no value, or perform significantly worse.



NOTE: While the performance penalty of negative shift checks may be alarming, the user can eliminate it entirely by modifying the call site as shown previously. When `s` is unsigned, `shl_bi_branching` and `shl_branching` are equivalent.

§ 4.4. Naming

We chose the names `shl` and `shr` because they are as short as possible, while still being familiar to many programmers. For example, the x86 and ARM instruction sets have shift instructions named `SHL` and `SHR`, Pascal has built-in operators with these names, and Rust uses `Shl` and `Shr` as the names of the traits that must be implemented for types to support the shift operators.

We hope the brevity will encourage adoption of `std::shl` and `std::shr` as safe alternatives to the built-in operators. These abbreviations are also consistent with the existing `std::rotl` and `std::rotr` functions, which are conspicuously not named `std::rotate_left` and `std::rotate_right`, respectively.

§ 4.5. Signatures

We propose the following signatures:

```
template<class T, class S>
constexpr T shl(T x, S s) noexcept;

template<class T, class S>
constexpr T shr(T x, S s) noexcept;
```

The rationale for these signature is as follows:

- We follow the design of `std::rotl` and `std::rotr` in proposing that the shift functions do not perform integer promotion.
- Like `rotl` and `rotr`, these functions don't participate in overload resolution unless `T` is an integer type. Unlike `rotl` and `rotr`, the proposed `shl` and `shr` accept either signed or unsigned integral types for the first operand: for `shr`, the signedness of the first parameter

determines whether an arithmetic or logical shift is performed, and it would be surprising if `shl` did not also accept signed types.



- Unlike `rotr` and `rotr`, the shift amount is not passed via `int`. That is because shifting by 2^{32} is not equivalent to shifting by 0, but such a large value stored in a `long long` may be turned into zero due to upper bits being discarded in the conversion. For rotations, this is not a problem because for power-of-two wide integers, discarding upper bits has no effect on the result.
- The functions are `noexcept` because they have a wide contract.

§ 4.6. SIMD support

Following [P2933R4], almost all `<bit>` functions have a corresponding overload in `<simd>`, including `std::rotr` and `std::rotr`.

The proposed functions should also have `<simd>` overloads, in the style of `simd::rotr` and `simd::rotr`. That is, overloads that can either shift a `simd::vec` by a `simd::vec` of shift amounts, or by a scalar shift amount which applies to all elements in the `simd::vec`.

§ 5. Possible implementation

The scalar `std::shl` and `std::shr` may be implemented as follows:

```
template<signed-or-unsigned T, signed-or-unsigned S>
constexpr T shl(T x, S s) noexcept {
    constexpr auto width = S(numeric_limits<make_unsigned_t<T>>::digits);
    if constexpr (is_signed_v<S>) {
        if (s < 0) {
            return s <= -width ? T(x < 0 ? -1 : 0) : x >> -s;
        }
    }
    return s >= width ? T(0) : x << s;
}

template<signed-or-unsigned T, signed-or-unsigned S>
constexpr T shr(T x, S s) noexcept {
    constexpr auto width = S(numeric_limits<make_unsigned_t<T>>::digits);
    if constexpr (is_signed_v<S>) {
        if (s < 0) {
            return s <= -width ? T(0) : x << -s;
        }
    }
    return s >= width ? T(x < 0 ? -1 : 0) : x >> s;
}
```

See also [Cxx26BitPermutations] for a reference implementation and test code.

`simd::shl` and `simd::shr` may be implemented (naively) by performing the operation for each element. A less naive implementation would be based on the branchless variants shown

in §4.3.2. Benchmarks of `std::shl` variants.



NOTE: Conveniently, the underlying instructions for comparing integers yield bit masks where every bit is set when the comparison succeeds. This makes it easy to implement `(s < 32) * (x << s)` in terms of e.g. `VPCMPGTD`, `VPAND`, and `VPSLLD` on x86.

§ 6. Wording

The following changes are relative to [\[N5014\]](#).



NOTE: The `shl` and `shr` functions should immediately precede the `rotr` and `rotr` functions in the wording. Therefore, if this paper and [\[P3764R0\]](#) are adopted in the same meeting, the `shl` and `shr` functions should be inserted between the `msb_to_mask` and `rotr` functions.

§ 6.1. `[version.syn]`

Bump feature-test macros in [\[version.syn\]](#) as follows:



```
#define __cpp_lib_bitops 201907L 20XXXXL // freestanding, also in <bit>
#define __cpp_lib_simd   202502L 20XXXXL // also in <simd>
```

§ 6.2. `[bit]`

In [\[bit.syn\]](#), change the synopsis as follows:



```
namespace std {
    [...]

    // [bit.shift], shifting
    template<class T, class S>
        constexpr T shl(T x, S s) noexcept;
    template<class T, class S>
        constexpr T shr(T x, S s) noexcept;

    // [bit.rotate], rotating
    template<class T>
        constexpr T rotr(T x, int s) noexcept;
    template<class T>
        constexpr T rotr(T x, int s) noexcept;

    [...]
}
```

In [\[bit\]](#), add a new subclause immediately preceding [\[bit.rotate\]](#):



Shifting

[bit.shift]

1 In the following descriptions, the result r of an arithmetic operation is first represented in a hypothetical integer type with sufficient range, then converted to T as if by `static_cast<T>(r)`.

2 [Note: The value of s in the following descriptions may be negative, which has the effect of “shifting in the opposite direction”. — *end note*]

```
template<class T, class S>
constexpr T shl(T x, S s) noexcept;
```

3 *Constraints:* Each of T and S is a signed or unsigned integer type ([basic.fundamental]).

4 *Returns:* $x \times 2^s$ rounded towards negative infinity.

```
template<class T, class S>
constexpr T shr(T x, S s) noexcept;
```

5 *Constraints:* Each of T and S is a signed or unsigned integer type ([basic.fundamental]).

6 *Returns:* $x \times 2^{-s}$ rounded towards negative infinity.

[Note: `shl(x, s)` is equivalent to `shr(x, -s)` if s is of signed type, except that $-s$ can overflow ([expr.pre], [expr.unary.op]). — *end note*]

§ 6.3. [simd]

In [simd.syn], change the synopsis as follows:



```
[...]

// [simd.bit], bit manipulation

[...]

template<simd-type V0, simd-type V1>
    constexpr V0 shl(const V0& v, const V1& s) noexcept;
template<simd-type V, class S>
    constexpr V  shl(const V& v, S s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 shr(const V0& v, const V1& s) noexcept;
template<simd-type V, class S>
    constexpr V  shr(const V& v, S s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 rotl(const V0& v, const V1& s) noexcept;
```



```

template<simd-type V>
    constexpr V  rotl(const V& v, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 rotr(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V  rotr(const V& v, int s) noexcept;

[...]

// See [simd.bit], bit manipulation
using simd::byteswap;
using simd::bit_ceil;
using simd::bit_floor;
using simd::has_single_bit;
using simd::shl;
using simd::shr;
using simd::rotr;
using simd::rotr;
using simd::bit_width;
using simd::countl_zero;
using simd::countl_one;
using simd::countr_zero;
using simd::countr_one;
using simd::popcount;

[...]

```

In [\[simd.bit\]](#), immediately preceding the first declaration of `rotr`, insert the following:



```

template<simd-type V0, simd-type V1>
    constexpr V0 shl(const V0& v0, const V1& v1) noexcept;
template<simd-type V0, simd-type V1>
    constexpr V0 shr(const V0& v0, const V1& v1) noexcept;

```

11 Constraints:

- The type `V0::value_type` is a signed or unsigned integer type ([\[basic.fundamental\]](#)),
- the type `V1::value_type` is a signed or unsigned integer type,
- `V0::size() == V1::size()` is `true`, and
- `sizeof(typename V0::value_type) == sizeof(typename V1::value_type)` is `true`.

12 Returns: A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(v0[i], v1[i])` for all i in the range `[0, V0::size())`, where `bit-func` has the same behavior as the corresponding scalar function from [<bit>](#).

```

template<simd-type V, class S>
    constexpr V shl(const V& v, S s) noexcept;

```

```
template<simd-type V, class S>
constexpr V shr(const V& v, S s) noexcept;
```



13 *Constraints*: Each of `V::value_type` and `S` is a signed or unsigned integer type ([basic.fundamental]).

14 *Returns*: A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(v[i], s)` for all i in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

§ 7. References

[N5014] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-08-05)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P2933R4] Daniel Towner, Ruslan Arutyunyan. Extend `<bit>` header function with overloads for `std::simd` (2025-02-13)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2933r4.html>

[P3643R0] Jan Schultke. `std::to_signed` and `std::to_unsigned` (2025-03-13)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3643r0.html>

[P3764R0] Jan Schultke. A utility function for propagating the most significant bit (2025-07-15)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3764r0.html>

[QuickBench] Benchmark for various ways of handling negative shift amounts

https://quick-bench.com/q/HcAUoOoGFHnm5nx1G_rJs52oLaY

[CompilerExplorer] Assembly comparison for various ways of handling negative shift amounts

<https://godbolt.org/z/1Won5qjo9>

[Cxx26BitPermutations] Jan Schultke.

<https://github.com/Eisenwave/cxx26-bit-permutations>